Shell Tutorial

September 25, 2025

1 Shell tutorial

In this tutorial we will show the basic shell commands and how to us them. You should try them out in an actual linux shell, like on the server. Running this in a notebook wouldn't work.

The shell environment is a robust non-point-and-click way of navigating and altering your computers environment. It is also very easy to access external servers via a terminal without the use of a browser application so that you can also work in another environment remotely.

1.0.1 Navigating your workspace

When you open a shell, you will be in a certain folder, usually `Home`. With the comand `mkdir` (which stands for *make directory*) we can make a folder.

[ ]:
```
mkdir testdir
```

Now we can move into that folder using `cd`

[ ]:
```
cd testdir
```

In this folder, we can create a file by opening an unexisting file with the `nano` command.

[ ]:
```
nano newfile.txt
```

This will open up a new 'screen' in the terminal, the nano editor. Let's make two comma separated columns of integers linking words. (Paste is `Ctrl + Shift + V`, try to look up how to copy)

[ ]:
```
1,one
2,two
3,three
4,foo
5,fizz
```

Save with `Ctrl + s` and close nano with `Ctrl + x`. You can open it again with `nano newfile.txt` or paste the contents on the terminal using `cat`. If you ever cat a large file or do an operation that you want to quit because it will take too long or crash, press `Ctrl + c`.

[ ]:
```
cat newfile.txt
```

```
1,one
2,two
3,three
```

```
4,foo
5,fizz
```

To see how many lines and characters a file has, use `wc`. Add `-l` for just the amount of rows.

[ ]:
```
wc -l newfile.txt
```

```
5 newfile.txt
```

To see all files in the current directory, use `ls`.

[ ]:
```
ls
```

```
newfile.txt
```

If a file is big, we can zip it using `gzip`.

[ ]:
```
gzip newfile.txt
```

To unzip, use `gunzip` or `gzip -d`

[ ]:
```
gunzip newfile.txt.gz
cat newfile.txt
```

```
1,one
2,two
3,three
4,foo
5,fizz
```

Using `gzip -d` yields the same result, but uses an option, namely `-d` for *decompress*. Run `gzip --help` for all the options. You can see that `-c` would write to standard output or stdout, which is the terminal environment, so it gets printed there. Try it. You can combine options, or tags, by concatenating them like `-dc` or `-cd`. Unzipping to stdout keeps the original file as well. Also try `zcat` as a replacement for `gzip -dc`.

[ ]:
```
gzip newfile.txt
gzip newfile.txt.gz -dc
```

```
1,one
2,two
3,three
4,foo
5,fizz
```

Let's put the output printed to stdout in a file with `>`. Files don't actually need extnsions like `.txt`.

[ ]:
```
gzip newfile.txt.gz -dc > copied
```

Now check which files are in this folder, but withh extra info per file, like size.

[ ]:
```
ls -l
```

```
total 8.0K   -rw-rw----+ 1 vsc35840 vsc35840 33 Sep 24 17:37 copied   -rw-rw----+ 1
vsc35840 vsc35840 65 Sep 24 17:22 newfile.txt.gz
```

The `-h` tag would print the file size in human readable format, so using M and G instead of just bytes. Adding `-a` would also show hidden files and folders which start with `.`.

[ ]:
```
mkdir .hidden
ls -a
```

```
.  ..  .hidden  copied  newfile.txt.gz
```

You may have noticed the `.` and `...` Those are this and the parent folder respectively. so `ls .` or `ls ./` are equivalent with `ls` and `ls ..` or `ls ../` shows everything in the parent folder.

[ ]:
```
ls .
ls ./
ls ..
ls ../
```

Copy a file with `cp`, but you need to give the copy a name. When typing `cp c` press tab to autofill to `cp copied`, this also works for the target file which doesn't yet exist.

[ ]:
```
cp copied copied.copy
```

Remember we made a file by *piping* our output from stdout into a file, let's put the contents of a file in a new file. But i just want the first two lines using `head`. Without using `-n`, head would print the first 10 lines. Look up which command shows the last 10 lines.

[ ]:
```
head copied -n 2 > copied_anothercopy.txt
```

Let's add a line `6,six` with `echo` and concat it to an existing file with `>>`.

[ ]:
```
echo '6,six' >> copied_anothercopy.txt
```

Let's remove the copies with `rm`.

[ ]:
```
rm copied copied.copy copied_anothercopy.txt
```

We could have removed them all at once using a *wildcard*: `*`. `copied*` refers to every file starting with `copied`. ALWAYS BE CAREFULL WITH `rm`, IT CAN'T BE UNDONE!!!

[ ]:
```
rm copied*
```

Let's go back a level

[ ]:
```
cd ../
```

Now let's move the file here with `mv`. You need to define a destination folder like `.`

[ ]:
```
mv testdir/newfile.txt.gz .
```

Remove the folder with the `-r` tag, try without the tag first.

[ ]:
```
rm testdir -r
```

1.0.2 Let's do more advanced tasks.

Search You can use `grep` to look for words in files. First unzip the file.

[ ]:
```
grep three newfile.txt
```

```
3,three
```

Use `-n` to paste the linenumber in the file as well.

[ ]:
```
grep three newfile.txt -n
```

```
3:3,three
```

Can't remember a command you recently used? Use grep in combination with history to find it.

[ ]:
```
history | grep copied
```

Replacing strings Now we will systematically alter files, for example exchanging characters in a file using `tr`. Look up how this command works and explain to yourself why `tr , "\t"` replaces all commas with tab characters (`\t`). `tr` can only handle stdout, not files. Why does `"\t"` need quotation marks and `,` not? You can forward the output of one command into another using the *pipe* symbol `|`.

[ ]:
```
cat newfile.txt | tr , "\t"
```

```
1   one
2   two
3   three
4   foo
5   fizz
```

More advanced replacements can be done using `sed`. Now we will use `zcat` instead of gzip to paste the contents of a zipped file to stdout.

[ ]:
```
sed 's/foo/four/g' newfile.txt
```

```
1,one
2,two
3,three
4,four
5,fizz
```

Now make a command that unzips the file to stdout, replaces all commas with tabs, replaces foo with four and fizz with five, all on one line.

[ ]:
```

```

AWK The most useful command probably is awk. With awk you can do an operation per line. What is `-F` doing here? What if you don't include `NR>1`? What is so special about `$0`? Try printing just `$0`.

[ ]:
```
awk -F',' 'NR>1 { print $0","$2"_"$1 }' newfile.txt
```

Sort, Join and Uniq Use `sort` to sort files. Let's make an unsorted file first. `\n` is the newline character. Why do we need the `-e` there?

[ ]:
```
echo -e '4,baba\n2,abab\n5,baba' > bb.txt
```

Now sort it based on the first column. Try it with `-k2`, what changed? First change all commas with tabs using `awk` or `tr`.

[ ]:
```
awk -F',' '{ print $1"\t"$2 }' bb.txt | sort
```

You can insert an output on the place that a file usually is referenced using `<()`.

[ ]:
```
sort <(cat bb.txt | tr , '\t') -k2
```

Now we will join the two files and pipe it somewhere new. Explain to yourself why this command looks this way.

[ ]:
```
join <(sort <(cat bb.txt | tr , '\t') -k1) <(cat newfile.txt | tr , '\t') -1 1␣
↪-2 1 > joined
```

```
2 abab two 4 baba foo 5 baba fizz
```

How many unique words are their in the second column? `uniq` will return the unique lines from a sorted file.

[ ]:
```
awk '{ print $2 }' joined | sort | uniq
```

```
abab baba
```